

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50% | Duration: 1 Hour | Total Marks: 50

ANNEXURE A

DESIGN TASK

Student 15: M ARSHINI | Register Number: 192424322

Design Task

Distributed Voice Assistant for Smart Devices

Construct a distributed AI voice assistant system using PySpark and RDD operations to process speech data streams from millions of smart devices. The platform supports large-scale speech-to-text and intent-recognition pipelines, while intelligent agents personalize responses, maintain multi-device context, and continuously improve recognition accuracy from user interactions.

Design Objective

The proposed solution combines edge devices, a distributed PySpark processing layer, AI model services, and cooperating intelligent agents. Speech is securely ingested, transformed into text, classified into intents, enriched with user context, and converted into an appropriate response. Historical interactions and feedback are processed in parallel for model improvement.

1. System Architecture

The architecture is organized into five layers. Smart devices capture speech; the ingestion layer authenticates and queues events; the PySpark cluster performs distributed preprocessing and analytics; AI services perform speech recognition and intent classification; and the agent layer manages context, personalization, dialogue and learning.

Layer	Main Components	Purpose
Device Layer	Smart speakers, phones, TVs, wearables	Capture speech and receive responses
Ingestion Layer	API Gateway, authentication, message queue	Securely receive and distribute events
Distributed Processing	PySpark, RDD partitions, feature pipeline	Process large interaction datasets in parallel
AI + Agent Layer	STT, intent model, context/personalization/dialogue agents	Understand requests and decide actions
Response + Learning	TTS, feedback store, model registry	Respond to users and improve future models

Major Components

- Device Gateway authenticates devices and forwards speech events with minimal metadata exposure.
- PySpark workers process partitions concurrently and prepare large training/analytics datasets.
- Speech-to-Text service converts audio into text with a confidence score.
- Intent service extracts the user goal and entities such as device, location and time.
- Agent Coordinator invokes the correct specialist agent and maintains the conversation flow.

2. PySpark and RDD Operations

RDDs provide a distributed representation of large collections of speech-derived records. Each RDD is divided into partitions, allowing worker nodes to transform independent records simultaneously. The RDD pipeline is mainly used for historical interaction processing, feature preparation, analytics and training-data updates.

RDD Operation	Purpose	Voice Assistant Example
map	Transform every record	Normalize transcript and metadata fields
filter	Remove unwanted records	Discard corrupted or low-confidence samples
flatMap	Expand records into many elements	Create tokens or n-gram features from transcripts
reduceByKey	Aggregate values by key	Count requests for each intent
groupByKey	Group records by key	Collect events for a device/session
partitionBy	Control partitioning	Keep user/device records consistently distributed
cache	Reuse frequently accessed data	Cache cleaned training data for repeated experiments

Distributed Processing Pipeline

Raw interaction RDD → validation/filtering → transcript normalization → feature extraction → intent-label preparation → aggregation → model-training dataset. When the same cleaned dataset is reused, caching reduces repeated computation. Key-based partitioning also helps reduce unnecessary data movement.

Large-Scale Model Updating

- New interaction records are added to the historical learning pool.
- `reduceByKey` identifies changing intent frequencies and common failure cases.
- Low-confidence and user-corrected interactions are prioritized for review.
- Training jobs run across partitions and the resulting model is evaluated before deployment.
- Model versions are monitored for intent accuracy, latency and drift after release.

3. Intelligent Agent Architecture

Specialized agents divide decision-making into manageable responsibilities. They communicate through an agent coordinator and shared context service, allowing new capabilities to be added without redesigning the entire assistant.

Agent	Responsibility	Typical Action
Device Agent	Represents the connected device	Authenticate and forward request
Speech Agent	Manages speech recognition	Select STT service and return transcript/confidence
Intent Agent	Understand user goal	Classify intent and extract entities
Context Agent	Maintain cross-device state	Retrieve recent conversation and active tasks
Personalization Agent	Adapt to user preferences	Select preferred language, routines and response style
Dialogue Agent	Control interaction	Ask clarification or invoke a service
Learning Agent	Use feedback for improvement	Collect corrections and prepare high-value samples

Agent Interaction Example

For the request “Turn on the bedroom lights and set them to 40 percent,” the Speech Agent produces the transcript. The Intent Agent detects a lighting-control intent and extracts bedroom and brightness values. The Context Agent identifies the user’s home and relevant devices. The Personalization Agent checks preferred routines, and the Dialogue Agent validates and sends the command to the home-control service. The assistant then returns a concise voice response. A correction from the user is captured by the Learning Agent.

Multi-Device Context

A shared context service stores session ID, recent intents, extracted entities, active task, preferred language and device capabilities. This lets a user continue a conversation across a speaker, television or wearable while avoiding duplicate command execution.

4. Data Flow and Processing Pipeline

1. Audio Capture — the device records a short speech segment.
2. Secure Ingestion — the gateway authenticates the device and places the event into a distributed queue.
3. Preprocessing — segmentation, noise handling and metadata checks are applied.
4. Speech-to-Text — the speech model generates transcript text and confidence.
5. Intent Recognition — intent and entities are extracted from the transcript.
6. Agent Coordination — context, personalization and dialogue agents choose the next action.
7. Service Execution — the requested smart-device or information service is invoked.
8. Response Generation — the result is converted into speech/text and returned to the user.
9. Feedback and Learning — corrections and failed requests enter the PySpark improvement pipeline.

Scalability, Fault Tolerance and Security

- Horizontal scaling adds worker nodes as event volume increases.
- RDD partitioning enables parallel computation over independent interaction records.
- RDD lineage allows lost partitions to be recomputed when a worker fails.
- Load balancing prevents one worker from becoming a processing bottleneck.
- Sensitive audio should be encrypted and raw recordings retained only when necessary.
- Device identifiers should be protected or separated from learning datasets where possible.
- Monitoring should track latency, recognition accuracy, confidence and model drift.

Industry Challenges

Real deployment must handle background noise, accents, multilingual speech, heterogeneous devices, network delay, privacy requirements and changing user vocabulary. Edge preprocessing can reduce latency, while centralized PySpark analytics can handle large historical datasets and periodic model improvement.

5. Innovation and Expected Benefits

The design combines distributed data processing with agent-based intelligence. Lightweight preprocessing can occur near devices for fast response, while PySpark handles large-scale historical analytics and learning workloads. Agents make the system modular so capabilities such as home automation, reminders, media control and information retrieval can be added independently.

Key Benefits

- Scales interaction processing to millions of smart devices.
- Uses parallel RDD transformations for large historical datasets.
- Provides personalized responses using context and preferences.
- Supports continuous improvement from user feedback.
- Maintains conversation continuity across multiple devices.
- Allows individual agents and services to be upgraded independently.

6. Proposed Enhancement

A privacy-aware federated learning layer can be added for intent recognition. Devices or edge gateways would train limited local updates instead of sending complete user interaction histories. Protected updates can be aggregated centrally, while the Learning Agent monitors model drift and decides when a new global model should be deployed. This can reduce exposure of raw voice data while still benefiting from diverse user interactions.

Conclusion

The proposed distributed voice assistant satisfies CO5 by integrating PySpark, RDD operations, scalable distributed processing and intelligent agents. RDD transformations support large-volume speech-derived data preparation and analytics, while agents manage speech understanding, context, personalization, dialogue and learning. The feedback loop enables continuous improvement as the platform grows.